CP 03 - Pipeline DevSecOps

Operação Blackout - Challenge 2026

Mateus Kalil - RM565098
Leonardo Lago Garroti - RM565963

FIAP - 2 de novembro de 2026

1. Aplicação e escopo

A esteira foi preparada para um único jogo: Operação Blackout, recebido na pasta exgame/public_html/blackout. O jogo da Leroy Merlin Academy apresenta situações de defesa cibernética em cinco cenários. A validação no navegador confirmou a abertura, o início da cena 3D e a navegação entre as opções do jogo.

O material recebido contém o build pronto do navegador. O JavaScript, o CSS e os ícones foram preservados com seus hashes SHA-256. Um servidor HTTP em Node.js entrega esses arquivos, aplica os cabeçalhos de segurança e oferece o endpoint /health. O progresso do jogo continua no localStorage. O portal PHP, o login, o ranking em nuvem e os recursos de voz do portal não fazem parte deste contêiner.

O projeto React original e seu lockfile não acompanharam a pasta. Por isso, o build desta entrega empacota o snapshot recebido; não recompila o React a partir de seus módulos originais. Essa limitação também afeta a análise das bibliotecas já embutidas no jogo.

2. Repositório e organização

Repositório público: https://github.com/09780987/cp03-operacao-blackout-devsecops

A publicação usa somente a conta GitHub 09780987, conforme solicitado. Os dois integrantes são identificados pelo nome e RM na documentação. O workflow está em .github/workflows/devsecops.yml. A raiz também contém Dockerfile, .dockerignore, package-lock.json, src, scripts, test e docs.

O README apresenta a execução local e via Docker, o diagrama da esteira e os links dos relatórios. O documento da esteira explica a configuração. O documento de observações registra os achados e as decisões. A pasta de evidências reúne as capturas de tela, os arquivos originais e os metadados das execuções.

3. As oito etapas

1. Verificação de segredos

O Gitleaks lê o histórico completo com fetch-depth 0, usa --redact e bloqueia ocorrências não excepcionadas. Há um único identificador de exceção para o valor fictício gerado no teste controlado; sua justificativa está registrada. O relatório da análise é publicado como artefato.

2. Análise estática

O Semgrep aplica p/default e p/owasp-top-ten ao código Node e aos scripts, em modo estrito. O cliente recebe esses dois conjuntos de regras e p/react sobre uma cópia formatada. Os achados bloqueiam a execução. Os avisos de tempo esgotado na análise do código compilado ficam visíveis e são contados no resumo. O relatório é publicado como artefato.

3. Dependências, testes e empacotamento

O comando npm ci instala as dependências a partir do arquivo de versões travadas. A esteira registra a árvore de dependências, executa npm audit e os dez testes, gera o diretório de distribuição e publica um pacote identificado pelo SHA do commit, acompanhado de SHA-256. A auditoria de vulnerabilidades é informativa; as falhas na preparação do pacote e nos testes bloqueiam a execução. O relatório e o pacote da aplicação são publicados como artefatos.

4. SCA do projeto

Trivy fs usa vuln, misconfig e secret, incluindo dependências de desenvolvimento. Gera um relatório completo e outro filtrado em HIGH/CRITICAL com bloqueio. Uma análise adicional lê o inventário parcial das bibliotecas identificadas no código compilado. Os relatórios são publicados como artefatos.

5. Imagem e registro

Docker Buildx constrói a imagem e a publica no GHCR com a tag do SHA. O nome fica em minúsculas; a promoção usa o digest. A etapa possui a permissão packages: write e registra a imagem, o digest e o manifesto no relatório de construção. A imagem inclui os dados de proveniência e a lista de componentes de software, ou SBOM.

6. Análise da imagem

Trivy image examina a imagem publicada pelo mesmo digest da etapa 5. O relatório completo preserva todas as severidades. O controle de qualidade bloqueia ocorrências HIGH/CRITICAL com correção disponível, usando --ignore-unfixed. As credenciais do registro entram por variáveis de ambiente. O relatório é publicado como artefato.

7. Homologação e análise dinâmica

A imagem verificada é iniciada na infraestrutura de execução do GitHub Actions, no ambiente de homologação, sem privilégios, com sistema de arquivos somente para leitura e sem capacidades adicionais do Linux. Após a verificação de saúde, o ZAP Baseline gera HTML, JSON e Markdown. Os achados são informativos para a revisão; falhas que impeçam a geração do relatório continuam bloqueando a execução. Os relatórios são publicados como artefatos.

8. Revisão e produção

O ambiente de produção exige aprovação e aceita somente a branch main. Após a aprovação, a API atualizou o serviço gratuito do Render com o digest analisado. A etapa confirmou o estado ativo, o digest, a resposta HTTP 200 e a versão correspondente ao commit. O relatório de produção foi publicado como artefato.

As etapas seguem a ordem definida pelas dependências. Cada etapa executada publica seu relatório mesmo em caso de falha, com if: always(). O resumo final depende das oito etapas, baixa os artefatos, conta os achados com jq e publica o resultado no resumo da execução e em um artefato próprio. Um resumo parcial adicional fica disponível antes da aprovação, para apoiar a revisão. Relatório ausente é apresentado como indisponível; uma etapa pulada não equivale a resultado limpo.

4. Resultados verificados

A execução 41557869924 concluiu as oito etapas e o resumo com sucesso no GitHub Actions. Os artefatos originais, os metadados e os prints desta execução acompanham a entrega.

Na execução usada como referência, Gitleaks ficou sem segredos não excepcionados, o Semgrep não apontou vulnerabilidades bloqueantes e o npm audit registrou zero vulnerabilidades nas dependências instaladas pelo lock desta entrega. Os dez testes passaram. Trivy fs, o inventário parcial e o gate da

imagem não apresentaram ocorrências HIGH/CRITICAL bloqueantes. O ZAP ficou sem alertas High ou Medium.

O Semgrep registrou três avisos de tempo esgotado no cliente compilado. Esses avisos limitam a cobertura e não foram apagados do JSON nem convertidos em uma afirmação de análise completa. O código-fonte React e o arquivo original de versões travadas são necessários para aprofundar essa verificação. O inventário CycloneDX contém somente as quatro versões identificáveis: React 19.2.6, React DOM 19.2.6, Three.js 0.184.0 e React Three Fiber 9.6.1.

5. Achados e correções

A primeira base do ambiente de execução apresentou seis ocorrências HIGH nos pacotes brace-expansion 5.0.9 e undici 6.28.0, distribuídos junto com o npm. Os identificadores foram CVE-2026-102276, CVE-2026-102278, CVE-2026-109383, CVE-2026-109869, CVE-2026-19534 e CVE-2026-87018. A correção removeu npm e Yarn do estágio final, pois o servidor só precisa do Node. A análise seguinte da imagem do Blackout confirmou zero ocorrências bloqueantes. Os relatórios antes e depois foram preservados.

No DAST, o mesmo jogo foi comparado em uma instância local sem cabeçalhos e na instância endurecida. Os alertas Medium de ausência de CSP e proteção contra clickjacking foram corrigidos, assim como nosniff, Permissions-Policy e os cabeçalhos de isolamento. A CSP não permite scripts externos nem execução inline de JavaScript. Os testes confirmam os cabeçalhos também nas respostas de erro.

Dois alertas Low foram analisados como falsos positivos: o IP 192.168.1.5 pertence aos pacotes de rede simulados de uma atividade do jogo; o número 1831565813 é uma constante do gerador numérico do Three.js, e não um timestamp do servidor. Permanecem dois alertas informativos, sobre no-store e a arquitetura de aplicação web moderna. Nenhum deles foi suprimido no relatório do ZAP.

6. Demonstração do controle de qualidade

Uma credencial estritamente fictícia foi criada na branch teste/quality-gate. A execução 41557834346 falhou no Gitleaks com um achado redigido; as etapas seguintes foram puladas e o resumo executou. O experimento local também mostrou que apagar apenas o arquivo no último commit mantém o bloqueio, porque o valor continua no histórico.

Como o checkout completo também traz as refs das branches de teste, o mesmo valor fictício passou a aparecer na main. A correção registra somente seu fingerprint exato em .gitleaksignore, com commit, caminho, regra e linha. O valor foi gerado localmente para o experimento e não foi emitido como token por um serviço. Outras credenciais e outros commits continuam sujeitos ao gate. A correção na main e os relatórios originais são mantidos para comparação.

A correção foi validada na main pela execução 41557869924, depois do bloqueio 41557834346. O histórico completo foi examinado com a exceção exata do teste, e as oito etapas chegaram ao fim.

7. Rastreabilidade e revisão

Commit da execução de referência: 5fc7e6f6abb3485e1f7d56398380b133640866df

Imagem: ghcr.io/09780987/cp03-operacao-blackout-devsecops:5fc7e6f6abb3485e1f7d56398380b133640866df

Digest: sha256:f038e8bd6c53cab7bf1e5c3a301ed719af201778bf4f5b719a5fd734a53c2a55

Execução: https://github.com/09780987/cp03-operacao-blackout-devsecops/actions/runs/41557869924

O ambiente de produção exige a conta 09780987 como revisora, permite a branch main e não permite ignorar as regras de proteção por decisão administrativa. A etapa consulta o histórico do GitHub antes do acesso à nuvem.

8. Lições aprendidas

1. Uma auditoria npm limpa no projeto não garante uma imagem limpa. Os pacotes vulneráveis estavam no npm da base.

2. Apagar um segredo apenas do último commit não resolve sua presença no histórico. O teste do Gitleaks confirmou isso.

3. Um código compilado e minificado pode esgotar o tempo da análise estática. Os avisos precisam acompanhar a interpretação do resultado.

4. Um inventário parcial não substitui o lockfile original. A cobertura das dependências do cliente precisa continuar explícita.

5. Alertas de IP e timestamp dependem do contexto. Neste jogo, eram conteúdo didático e uma constante matemática.

6. SHA, digest, relatórios e aprovação ajudam a rastrear o que foi analisado e o que chegou ao deploy.

7. O resumo precisa funcionar também em falhas iniciais. A validação real revelou o formato diferente de download quando existe somente um artefato.

9. Entrega e reprodução

O ZIP reúne o projeto, este relatório simples, a documentação, os relatórios brutos, os prints e uma cópia portátil da imagem. Os arquivos temporários, caches, node_modules e configurações privadas do portal ficaram fora do pacote. Os arquivos do jogo recebido podem ser conferidos contra config/origem-jogo.json.

Os dez prints da entrega aparecem após as referências, identificados pelo título. A pasta de evidências também reúne as imagens originais separadamente.

Para executar pelo código: npm ci, npm test, npm run build e npm start. Para executar com Docker: construir o Dockerfile e publicar a porta 3000 somente em 127.0.0.1. A cópia portátil permite carregar a imagem com docker load, sem precisar reconstruí-la. As instruções completas e os links das execuções ficam no README.

A entrega está concluída com execução completa, teste de bloqueio e correção, aprovação registrada, imagem pública e jogo em produção. O ZIP inclui as evidências reais e a imagem portátil. Os nomes e RMs foram mantidos por solicitação da dupla; o histórico público usa e-mail protegido do GitHub.

Referências

Enunciado CP 03 - Pipeline DevSecOps, FIAP.

GitHub Actions: https://docs.github.com/actions

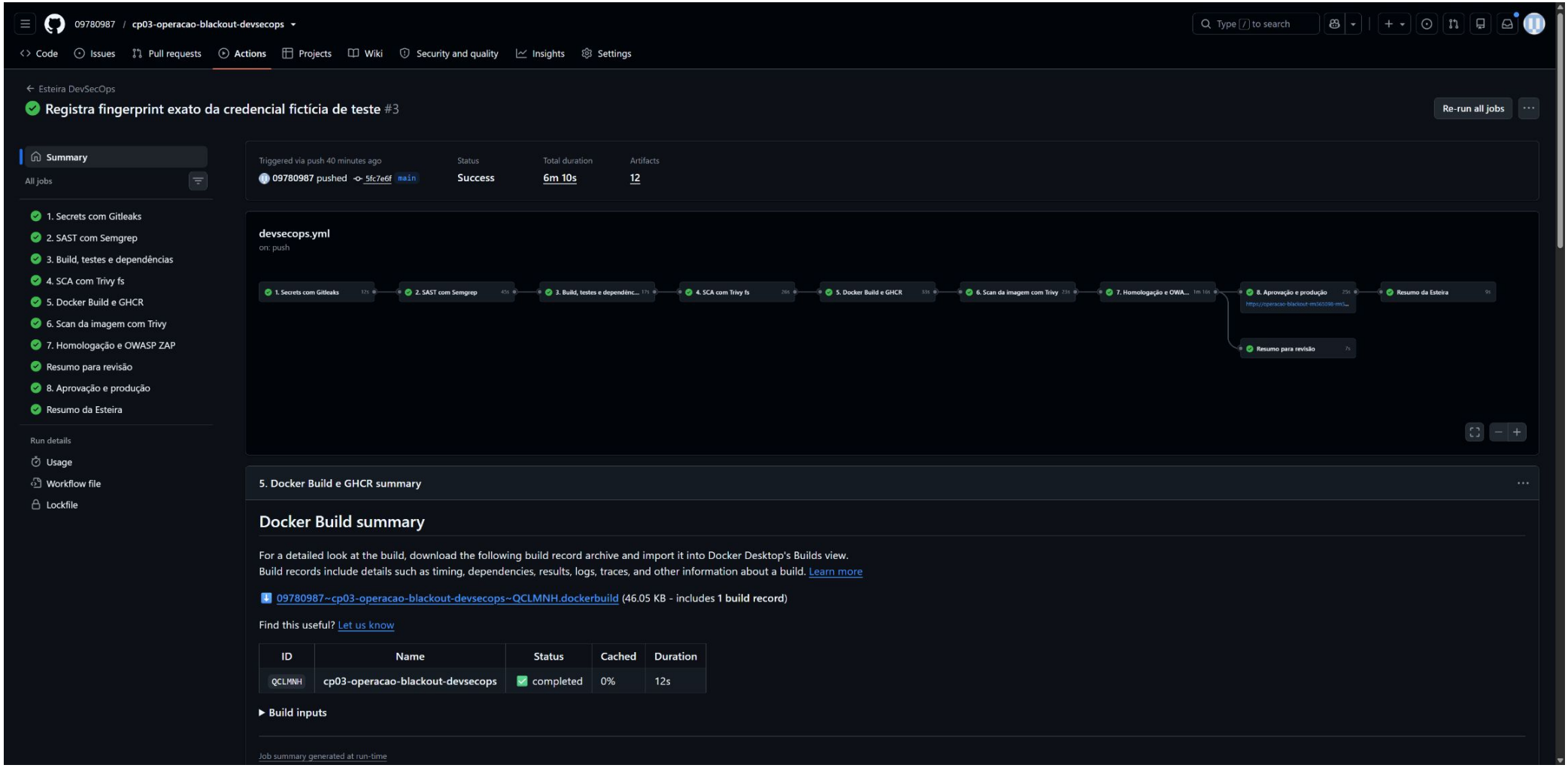
Gitleaks: https://github.com/gitleaks/gitleaks

Semgrep: https://semgrep.dev/docs

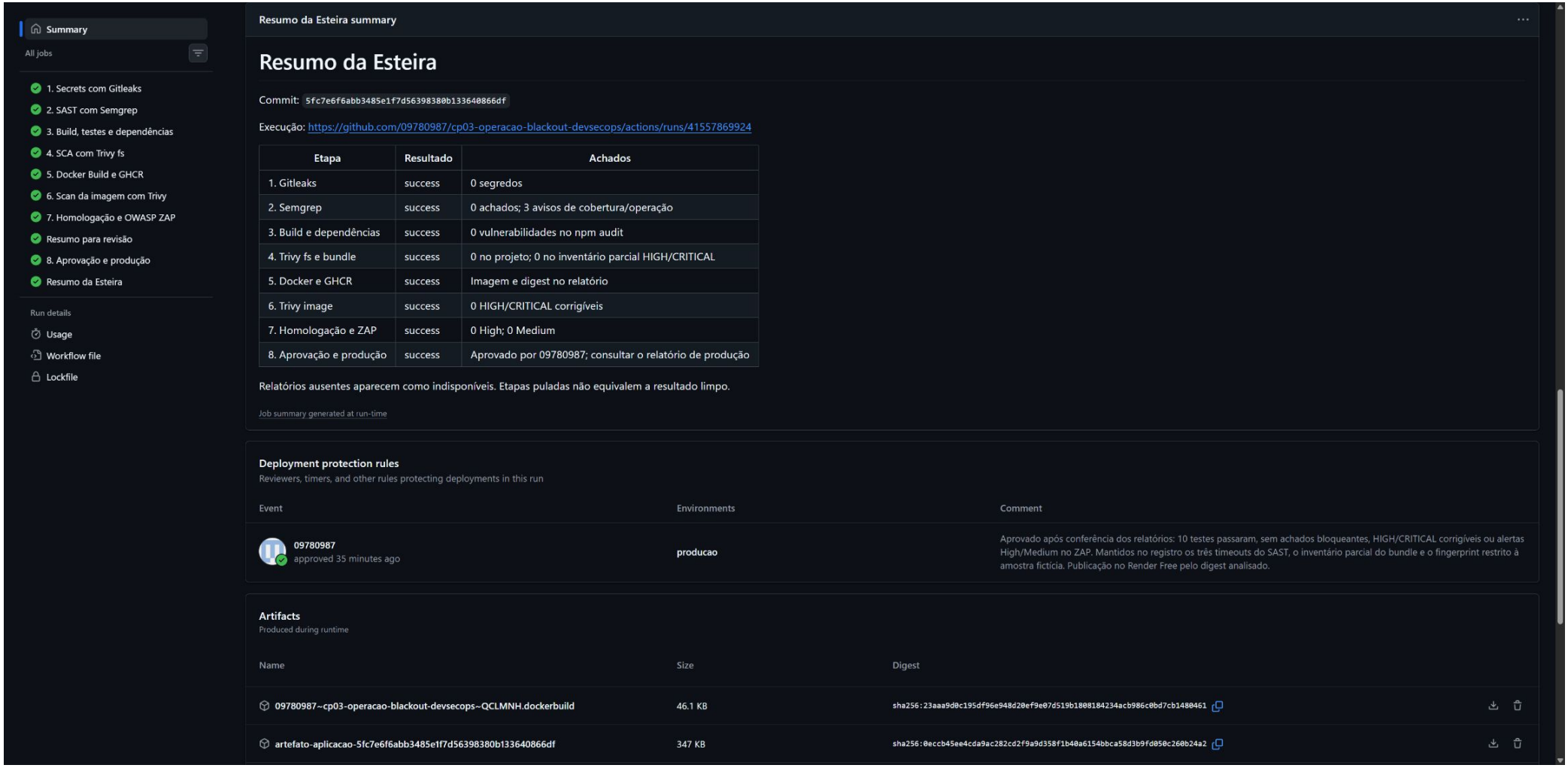
Trivy: https://trivy.dev

OWASP ZAP: https://www.zaproxy.org/docs/docker/baseline-scan/

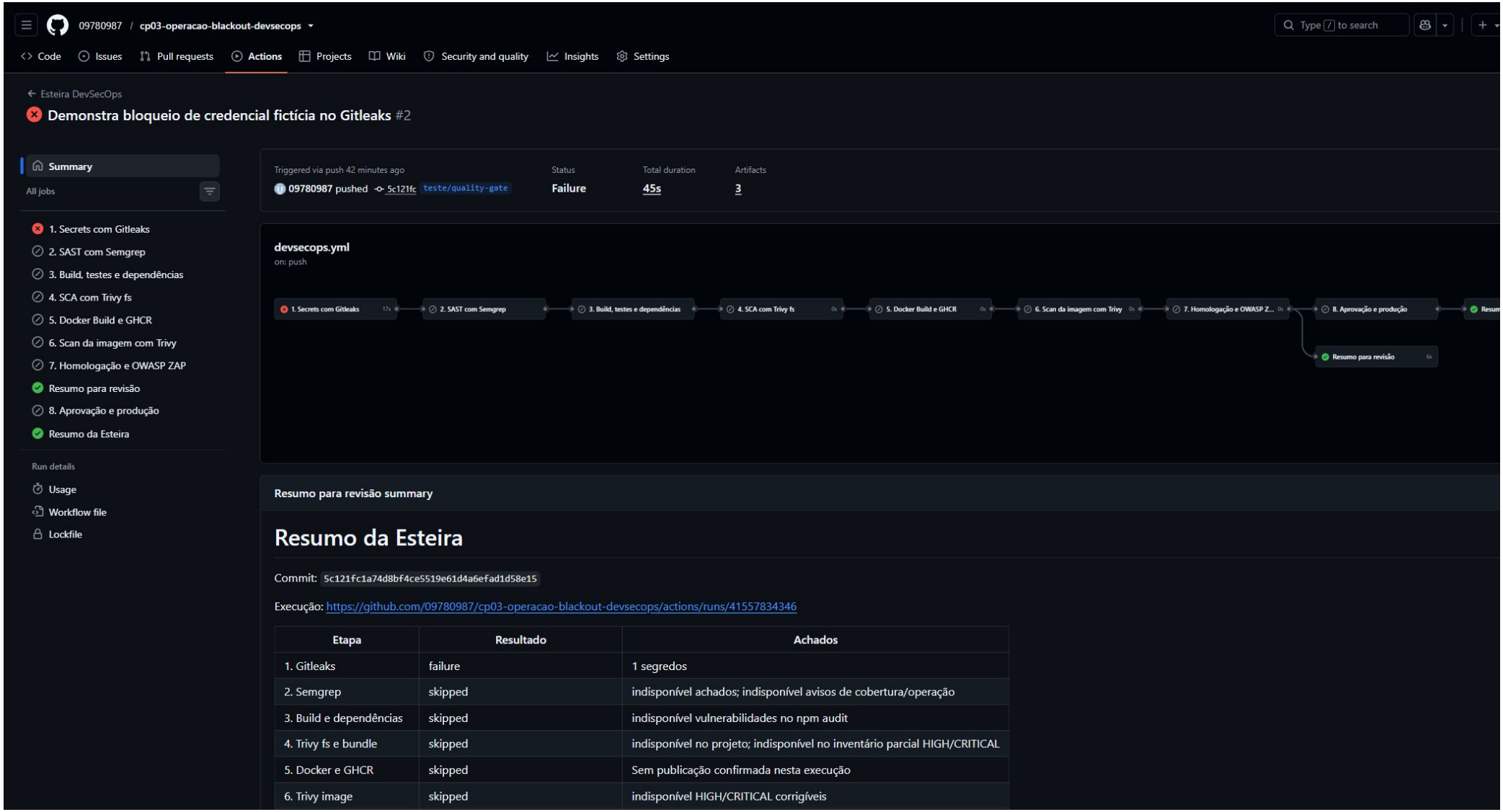
Execução completa da esteira



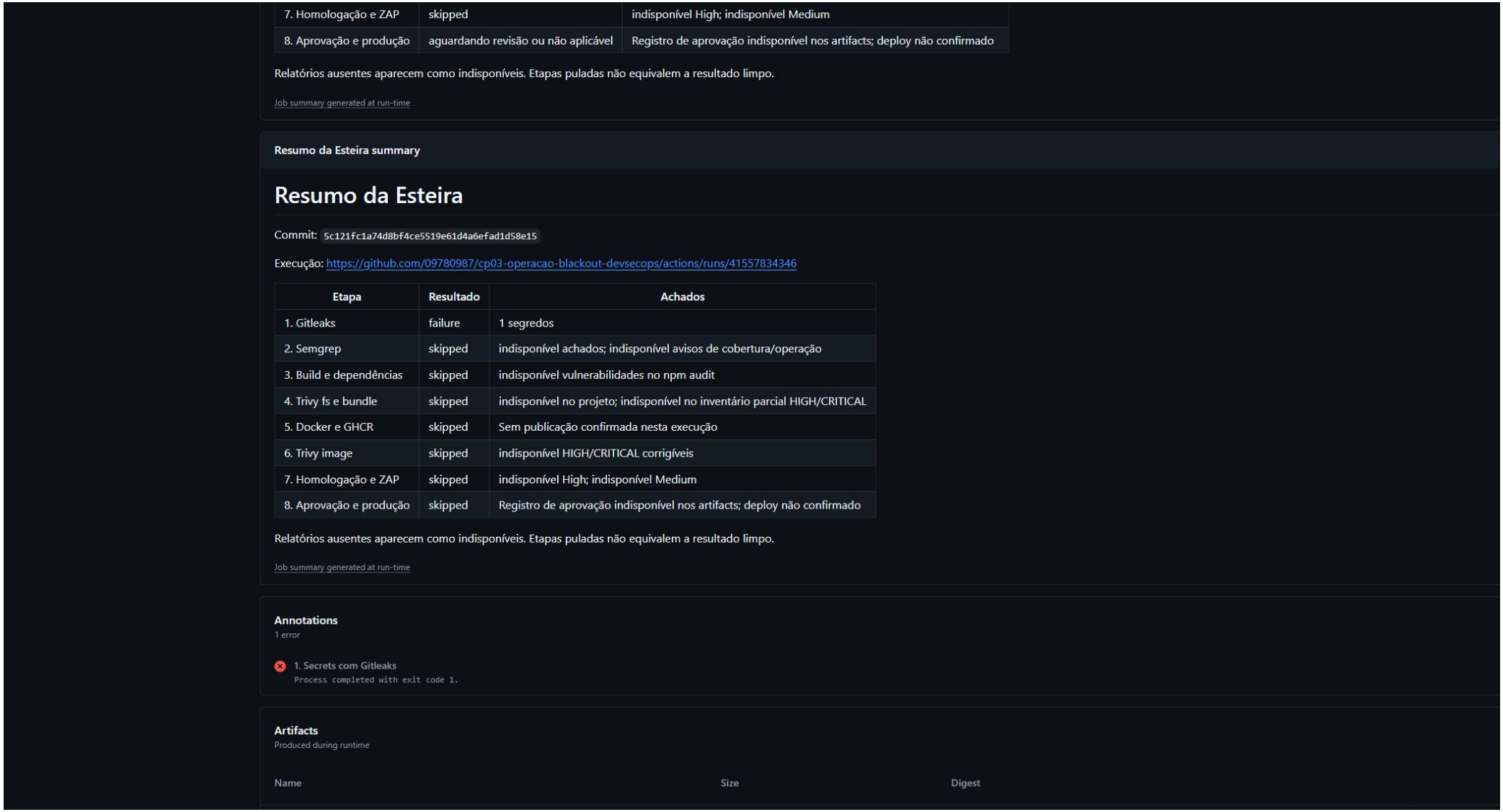
Resumo da Esteira



Execução bloqueada pelo controle de qualidade



Execução bloqueada pelo controle de qualidade - continuação



7. Homologação e ZAP	skipped	indisponível High; indisponível Medium
8. Aprovação e produção	aguardando revisão ou não aplicável	Registro de aprovação indisponível nos artifacts; deploy não confirmado

Relatórios ausentes aparecem como indisponíveis. Etapas puladas não equivalem a resultado limpo.

Job summary generated at run-time

Resumo da Esteira summary

Resumo da Esteira

Commit: `5c121fc1a74d8bf4ce5519e61d4a6efad1d58e15`

Execução: https://github.com/09780987/cp03-operacao-blackout-devsecops/actions/runs/41557834346

Etapa	Resultado	Achados
1. Gitleaks	failure	1 segredos
2. Semgrep	skipped	indisponível achados; indisponível avisos de cobertura/operação
3. Build e dependências	skipped	indisponível vulnerabilidades no npm audit
4. Trivy fs e bundle	skipped	indisponível no projeto; indisponível no inventário parcial HIGH/CRITICAL
5. Docker e GHCR	skipped	Sem publicação confirmada nesta execução
6. Trivy image	skipped	indisponível HIGH/CRITICAL corrigíveis
7. Homologação e ZAP	skipped	indisponível High; indisponível Medium
8. Aprovação e produção	skipped	Registro de aprovação indisponível nos artifacts; deploy não confirmado

Relatórios ausentes aparecem como indisponíveis. Etapas puladas não equivalem a resultado limpo.

Job summary generated at run-time

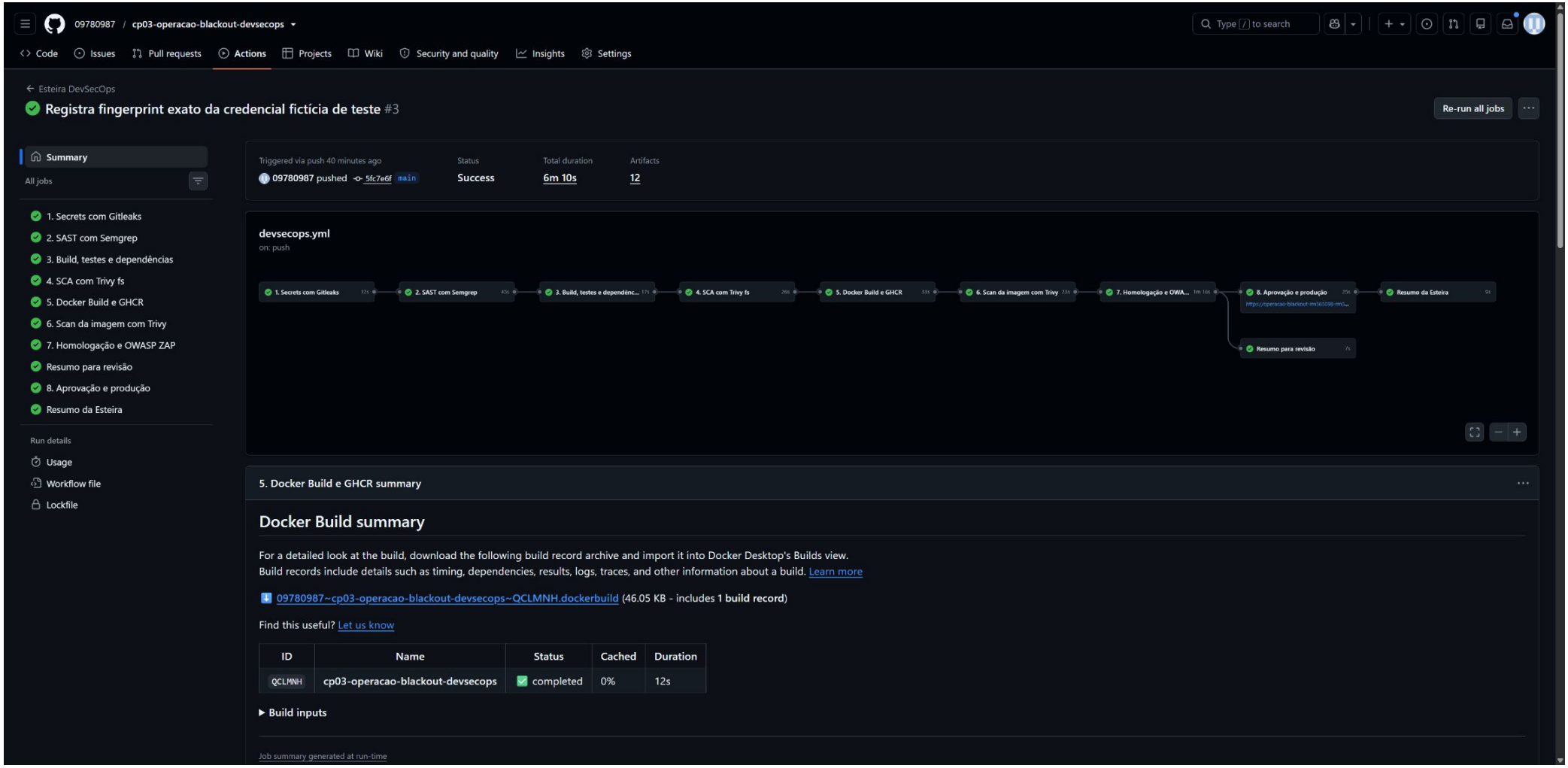
Annotations
1 error

1. Secrets com Gitleaks
`Process completed with exit code 1.`

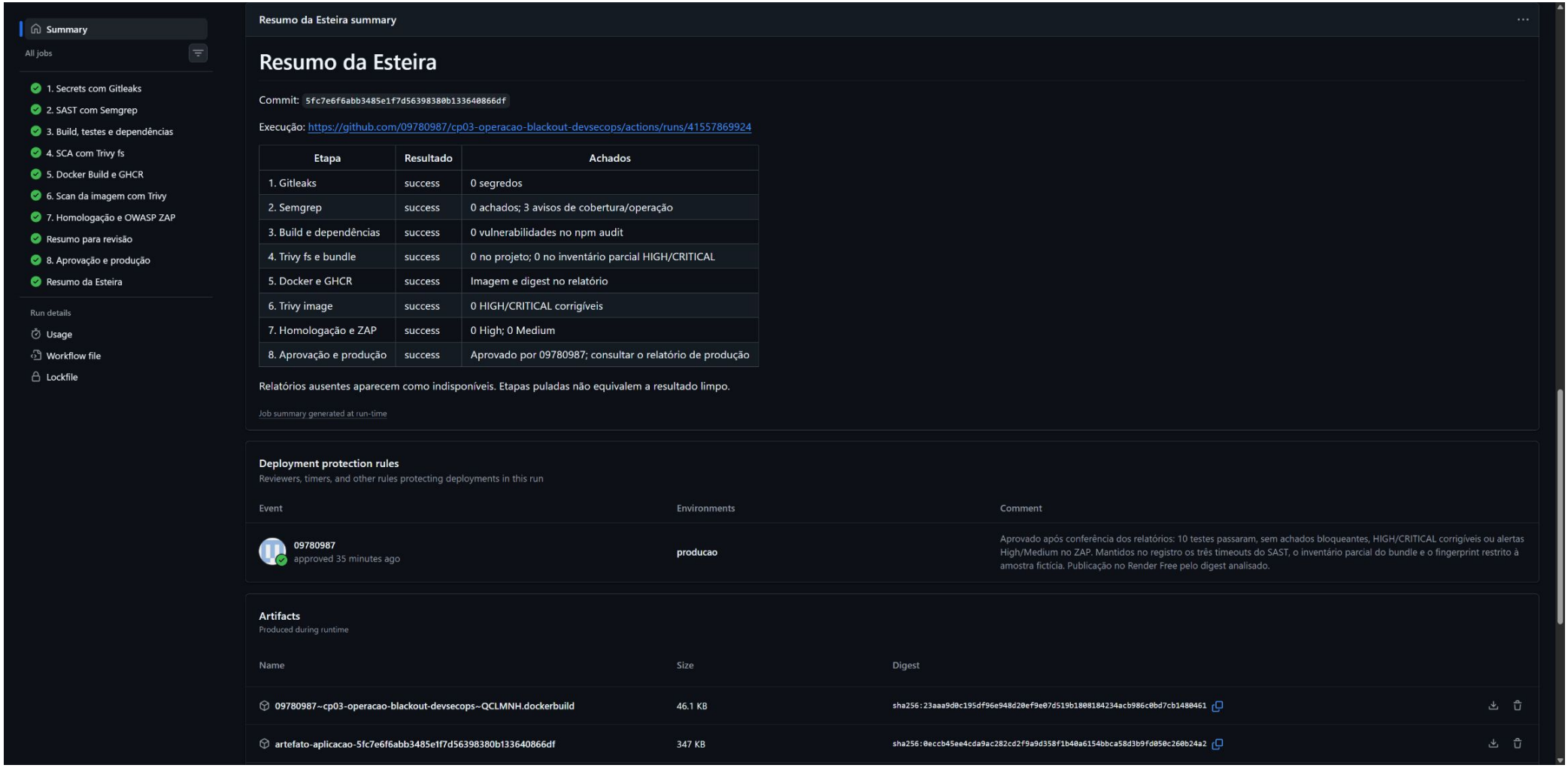
Artifacts
Produced during runtime

Name	Size	Digest

Execução após a correção do gate



Aprovação da produção



Configuração do ambiente de produção

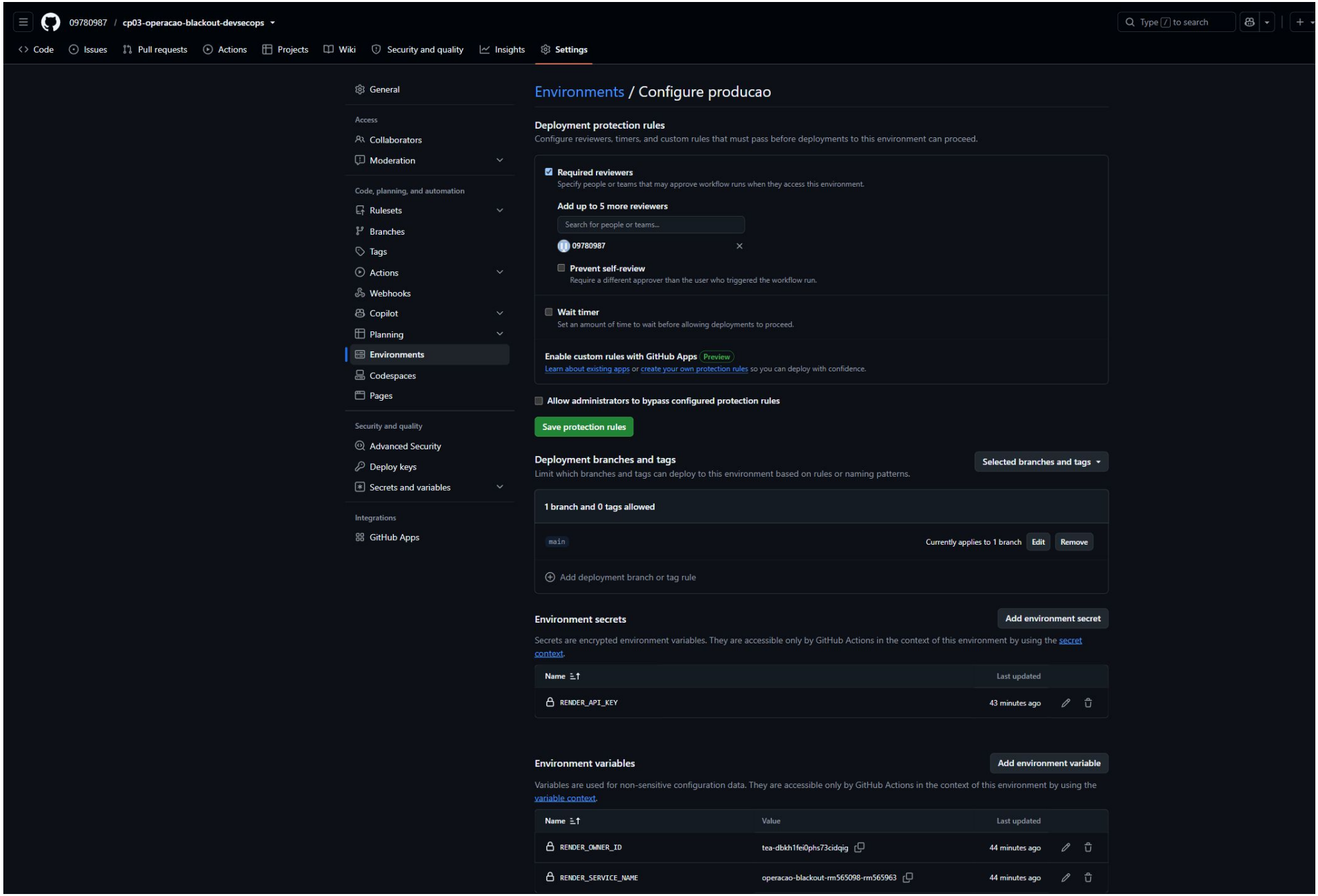
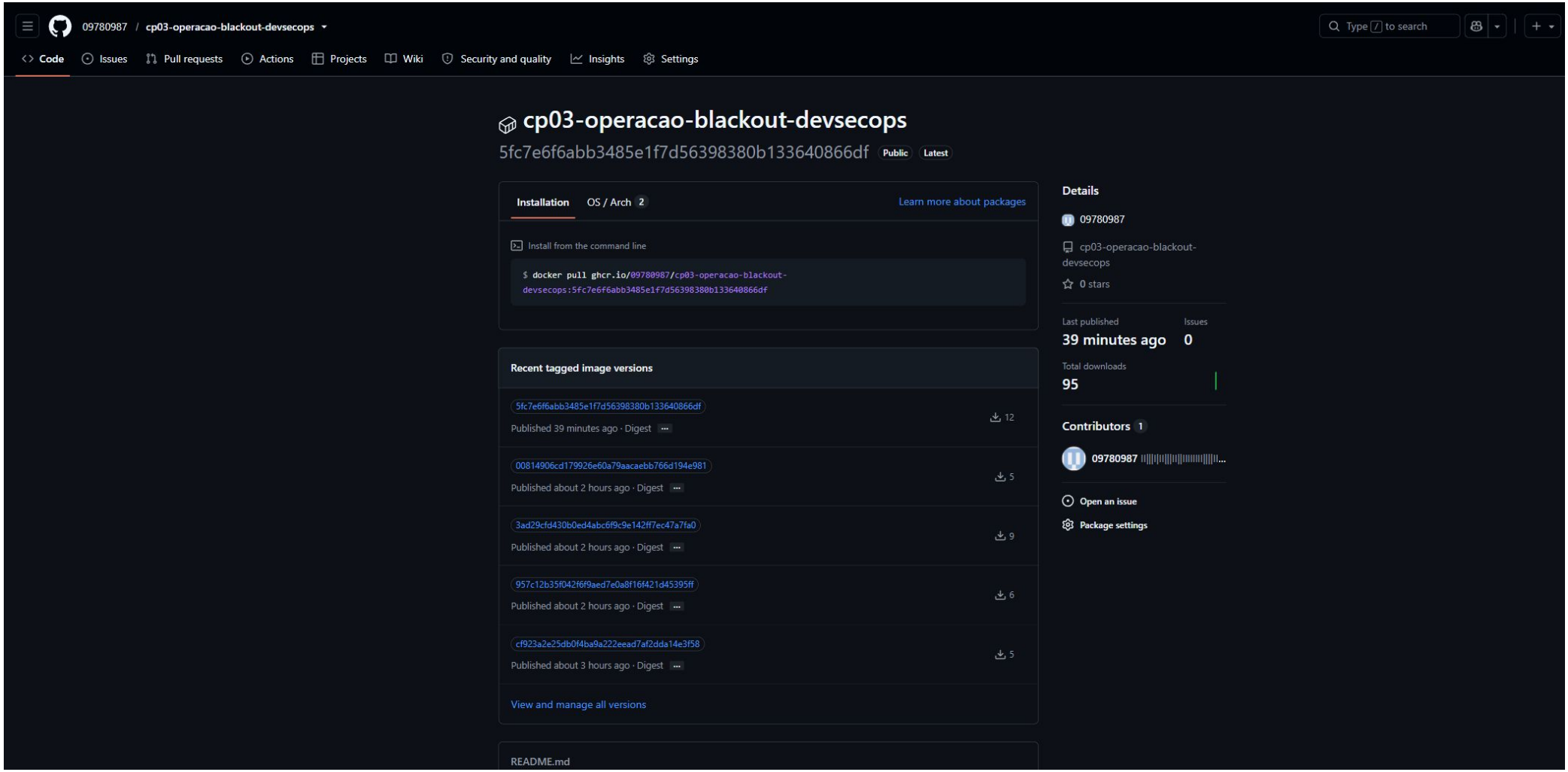
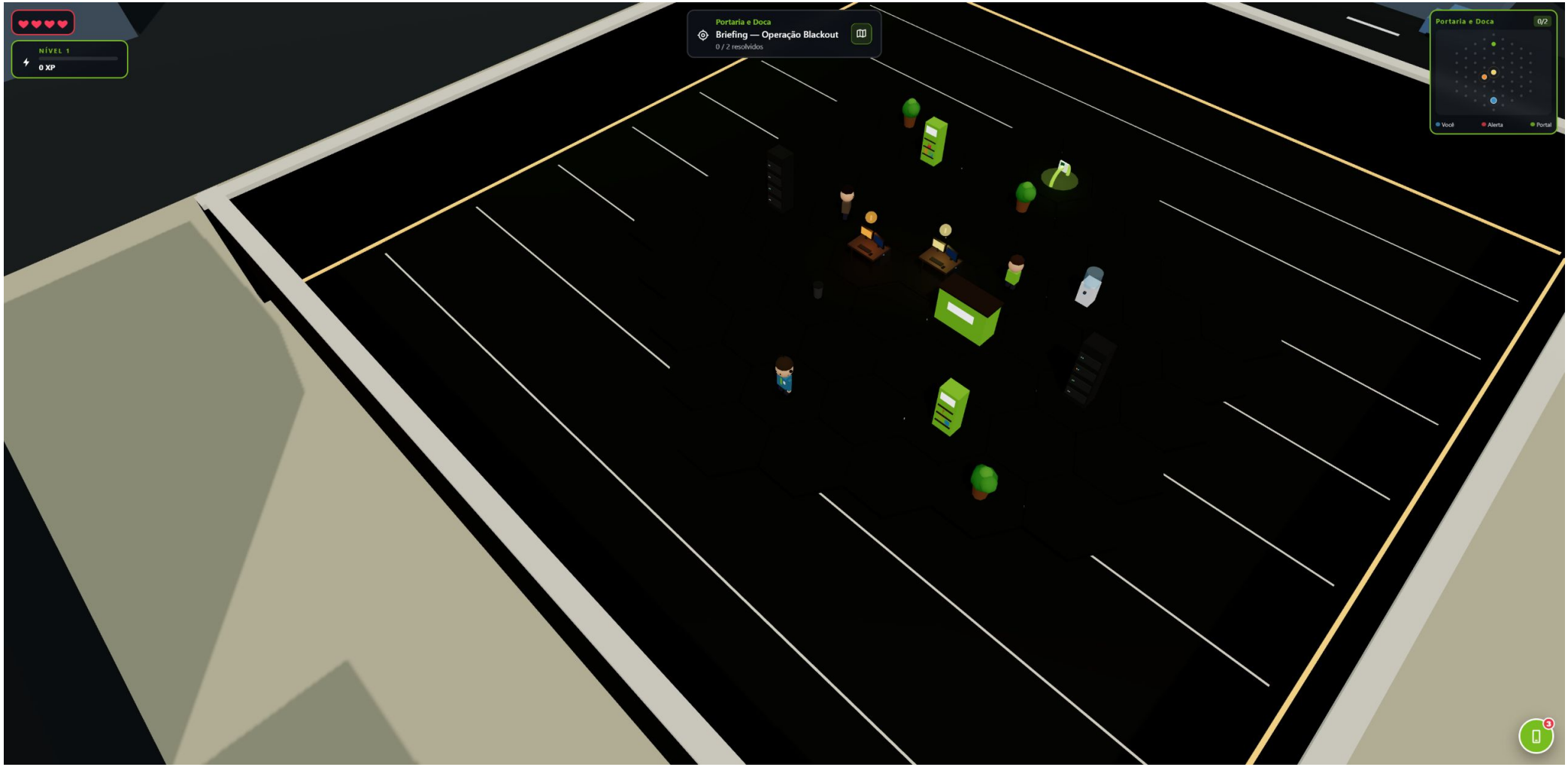


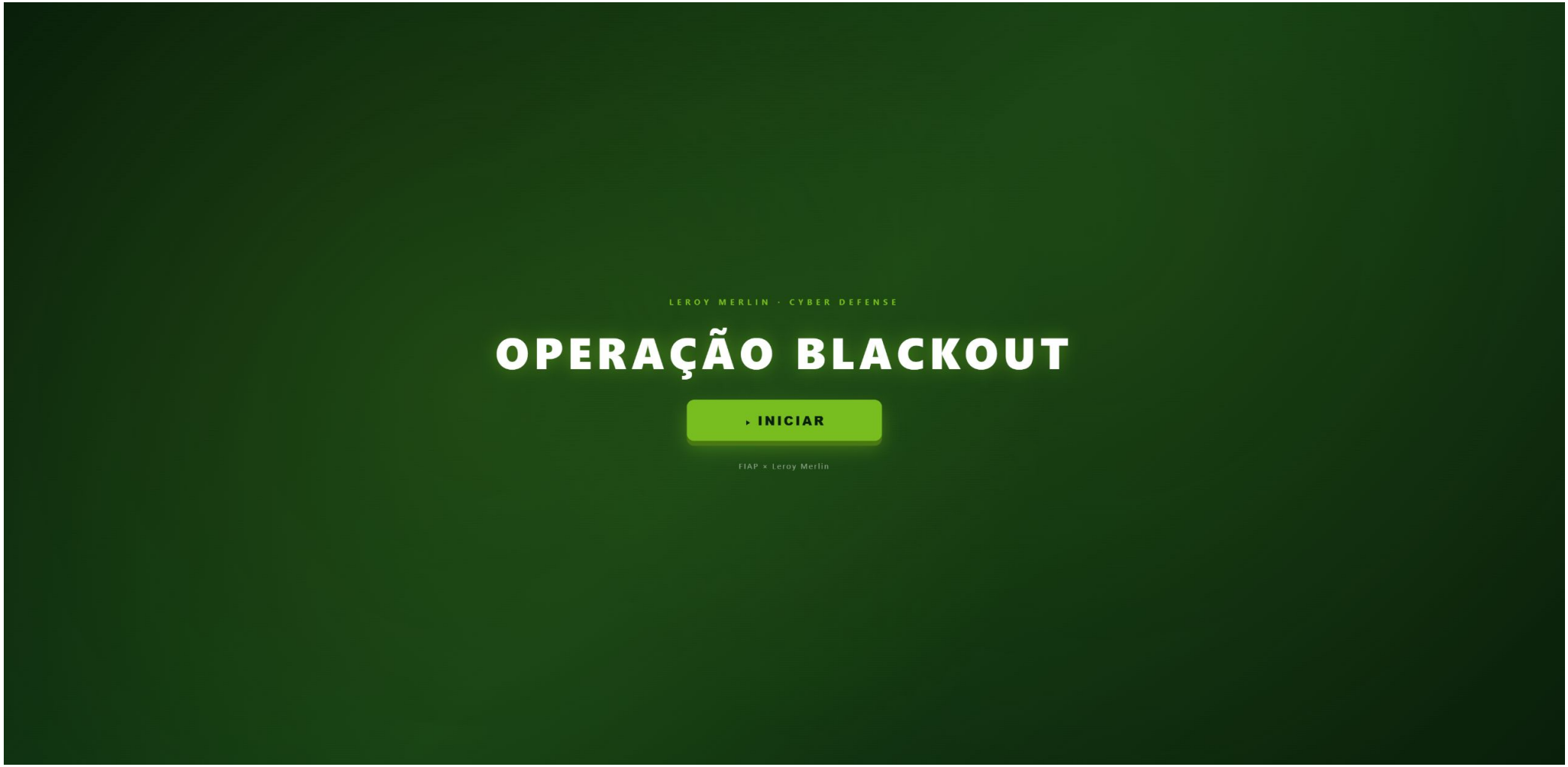
Imagem publicada em Packages



Jogo em produção



Tela inicial da Operação Blackout



Cenário Portaria e Doca

